

Divide and Conquer – Searching & Basic Problems

Binary Search, Finding Maximum and Minimum using Divide and Conquer

Program 1: Search an Element in a Sorted Array (Binary Search)

AIM

To search for a given key element in a sorted array using the Binary Search (Divide and Conquer) technique and print its position, or report that it is not found.

ALGORITHM

1. Start with low = 0 and high = n-1.
2. Repeat while low <= high:
3. Compute mid = (low + high) / 2.
4. If a[mid] equals key, return position (mid+1) and stop.
5. If a[mid] < key, set low = mid + 1 (search right half).
6. Else set high = mid - 1 (search left half).
7. If the loop ends without a match, report 'Element not found'.

PROGRAM (C)

```
#include <stdio.h>

int binarySearch(int a[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (a[mid] == key) return mid + 1;    // 1-based position
        else if (a[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter sorted elements: ");
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    int key;
    printf("Enter key to search: ");
    scanf("%d", &key);

    int pos = binarySearch(a, n, key);
    if (pos != -1)
        printf("Element found at position %d\n", pos);
    else
        printf("Element not found\n");
    return 0;
}
```

INPUT

```
5
10 20 30 40 50
```

30

(Test 2)

5

10 20 30 40 50

25

OUTPUT

Element found at position 3

(Test 2)

Element not found

RESULT

The program was executed successfully and the key element was located using Binary Search in $O(\log n)$ time, matching the expected output for both test cases.

Program 2: Find the Index of a Given Number (0-based Binary Search)

AIM

To find the 0-based index of a target number in a sorted array using Binary Search, returning -1 if the target is absent.

ALGORITHM

1. Set $low = 0$, $high = n-1$.
2. While $low \leq high$, compute $mid = (low+high)/2$.
3. If $a[mid] == target$, return mid as the index.
4. If $a[mid] < target$, move to the right half ($low = mid+1$).
5. Else move to the left half ($high = mid-1$).
6. If not found, return -1.

PROGRAM (C)

```
#include <stdio.h>
```

```
int findIndex(int a[], int n, int target) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (a[mid] == target) return mid;
        else if (a[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
```

```
int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter sorted elements: ");
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    int target;
    printf("Enter target: ");
    scanf("%d", &target);
```

```
int idx = findIndex(a, n, target);
printf("Index = %d\n", idx);
return 0;
}
```

INPUT

```
6
2 4 6 8 10 12
8
```

```
(Test 2)
6
2 4 6 8 10 12
5
```

OUTPUT

```
Index = 3
```

```
(Test 2)
Index = -1
```

RESULT

The program correctly returned the 0-based index of the target element or -1 when the element was absent, verifying both test cases.

Program 3: Count the Number of Iterations in Binary Search

AIM

To perform Binary Search on a sorted array and count the number of iterations taken to find the key or conclude it is absent.

ALGORITHM

1. Initialize low = 0, high = n-1, iterations = 0.
2. While low <= high, increment iterations by 1.
3. Compute mid = (low+high)/2.
4. If a[mid] == key, print 'Element found' with iteration count and stop.
5. If a[mid] < key, low = mid+1, else high = mid-1.
6. If the loop ends without success, print 'Element not found' with iteration count.

PROGRAM (C)

```
#include <stdio.h>

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter sorted elements: ");
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    int key;
    printf("Enter key: ");
    scanf("%d", &key);
```

```

int low = 0, high = n - 1, iterations = 0, found = -1;
while (low <= high) {
    iterations++;
    int mid = (low + high) / 2;
    if (a[mid] == key) { found = mid; break; }
    else if (a[mid] < key) low = mid + 1;
    else high = mid - 1;
}

if (found != -1)
    printf("Element found\nIterations = %d\n", iterations);
else
    printf("Element not found\nIterations = %d\n", iterations);
return 0;
}

```

INPUT

```

7
5 10 15 20 25 30 35
25

```

```

(Test 2)
7
5 10 15 20 25 30 35
40

```

OUTPUT

```

Element found
Iterations = 2

```

```

(Test 2)
Element not found
Iterations = 3

```

RESULT

The program successfully tracked and displayed the number of iterations required by Binary Search for both a successful and an unsuccessful search.

Program 4: Find the First Occurrence of an Element

AIM

To find the index of the first occurrence of a key in a sorted array that may contain duplicate elements, using a modified Binary Search.

ALGORITHM

1. Set low = 0, high = n-1, result = -1.
2. While low <= high, compute mid = (low+high)/2.
3. If a[mid] == key, store mid in result and continue searching the left half (high = mid-1) to look for an earlier occurrence.
4. If a[mid] < key, low = mid+1, else high = mid-1.
5. After the loop, if result != -1 print the first occurrence index, else print 'Element not found'.

PROGRAM (C)

```
#include <stdio.h>
```

```

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter sorted elements: ");
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    int key;
    printf("Enter key: ");
    scanf("%d", &key);

    int low = 0, high = n - 1, result = -1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (a[mid] == key) { result = mid; high = mid - 1; }
        else if (a[mid] < key) low = mid + 1;
        else high = mid - 1;
    }

    if (result != -1)
        printf("First occurrence at index %d\n", result);
    else
        printf("Element not found\n");
    return 0;
}

```

INPUT

8
1 2 2 2 3 4 5 6
2

(Test 2)
8
1 2 2 2 3 4 5 6
7

OUTPUT

First occurrence at index 1

(Test 2)
Element not found

RESULT

The modified Binary Search correctly located the first occurrence of the duplicate key and correctly reported absence for a key not present.

Program 5: Find the Last Occurrence of an Element

AIM

To find the index of the last occurrence of a key in a sorted array containing duplicates, using a modified Binary Search.

ALGORITHM

1. Set low = 0, high = n-1, result = -1.
2. While low <= high, compute mid = (low+high)/2.

3. If $a[mid] == key$, store mid in result and continue searching the right half ($low = mid+1$) to look for a later occurrence.
4. If $a[mid] < key$, $low = mid+1$, else $high = mid-1$.
5. After the loop, if $result \neq -1$ print the last occurrence index, else print 'Element not found'.

PROGRAM (C)

```
#include <stdio.h>

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter sorted elements: ");
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    int key;
    printf("Enter key: ");
    scanf("%d", &key);

    int low = 0, high = n - 1, result = -1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (a[mid] == key) { result = mid; low = mid + 1; }
        else if (a[mid] < key) low = mid + 1;
        else high = mid - 1;
    }

    if (result != -1)
        printf("Last occurrence at index %d\n", result);
    else
        printf("Element not found\n");
    return 0;
}
```

INPUT

```
8
1 2 2 2 3 4 5 6
2
```

```
(Test 2)
8
1 2 2 2 3 4 5 6
9
```

OUTPUT

Last occurrence at index 3

```
(Test 2)
Element not found
```

RESULT

The program accurately identified the last occurrence index of a repeated key and correctly handled the case where the key was absent.

Program 6: Highest and Lowest Temperature Recorded (Divide and Conquer)

AIM

To find the minimum and maximum temperature recorded over a week using the Divide and Conquer min-max algorithm.

ALGORITHM

1. If the segment has one element, it is both the min and max.
2. If the segment has two elements, compare them directly to get min and max.
3. Otherwise, divide the array into two halves at $\text{mid} = (\text{low} + \text{high}) / 2$.
4. Recursively find (min, max) of the left half and the right half.
5. Combine: overall min = smaller of the two mins; overall max = larger of the two maxes.
6. Return the combined (min, max) pair.

PROGRAM (C)

```
#include <stdio.h>

struct Pair { int minVal, maxVal; };

struct Pair minMax(int a[], int low, int high) {
    struct Pair result, left, right;
    if (low == high) {
        result.minVal = result.maxVal = a[low];
        return result;
    }
    if (high == low + 1) {
        if (a[low] < a[high]) { result.minVal = a[low]; result.maxVal = a[high]; }
        else { result.minVal = a[high]; result.maxVal = a[low]; }
        return result;
    }
    int mid = (low + high) / 2;
    left = minMax(a, low, mid);
    right = minMax(a, mid + 1, high);

    result.minVal = (left.minVal < right.minVal) ? left.minVal : right.minVal;
    result.maxVal = (left.maxVal > right.maxVal) ? left.maxVal : right.maxVal;
    return result;
}

int main() {
    int n;
    printf("Enter number of readings: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter values: ");
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);

    struct Pair p = minMax(a, 0, n - 1);
    printf("Minimum Temperature = %d\n", p.minVal);
    printf("Maximum Temperature = %d\n", p.maxVal);
    return 0;
}
```

INPUT

7
32 28 35 25 31 29 37

OUTPUT

Minimum Temperature = 25
Maximum Temperature = 37

RESULT

The Divide and Conquer approach correctly identified the minimum and maximum temperature values using only about $1.5n$ comparisons.

Program 7: Best and Worst Student Marks (Divide and Conquer)

AIM

To determine the highest and lowest marks scored by students in a class using the Divide and Conquer min-max algorithm.

ALGORITHM

1. If the segment has one element, it is both the min and max.
2. If the segment has two elements, compare them directly to get min and max.
3. Otherwise, divide the array into two halves at $\text{mid} = (\text{low} + \text{high}) / 2$.
4. Recursively find (min, max) of the left half and the right half.
5. Combine: overall min = smaller of the two mins; overall max = larger of the two maxes.
6. Return the combined (min, max) pair.

PROGRAM (C)

```
#include <stdio.h>

struct Pair { int minVal, maxVal; };

struct Pair minMax(int a[], int low, int high) {
    struct Pair result, left, right;
    if (low == high) {
        result.minVal = result.maxVal = a[low];
        return result;
    }
    if (high == low + 1) {
        if (a[low] < a[high]) { result.minVal = a[low]; result.maxVal = a[high]; }
        else { result.minVal = a[high]; result.maxVal = a[low]; }
        return result;
    }
    int mid = (low + high) / 2;
    left = minMax(a, low, mid);
    right = minMax(a, mid + 1, high);

    result.minVal = (left.minVal < right.minVal) ? left.minVal : right.minVal;
    result.maxVal = (left.maxVal > right.maxVal) ? left.maxVal : right.maxVal;
    return result;
}

int main() {
    int n;
    printf("Enter number of marks: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter values: ");
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);

    struct Pair p = minMax(a, 0, n - 1);
```



```
printf("Minimum Mark = %d\n", p.minVal);
printf("Maximum Mark = %d\n", p.maxVal);
return 0;
}
```

INPUT

8
78 92 65 88 95 72 81 69

OUTPUT

Minimum Mark = 65
Maximum Mark = 95

RESULT

The program correctly identified the lowest and highest marks in the class using the Divide and Conquer technique.

Program 8: Stock Price Analysis (Divide and Conquer)

AIM

To find the minimum and maximum stock prices recorded during a trading week using the Divide and Conquer approach.

ALGORITHM

1. If the segment has one element, it is both the min and max.
2. If the segment has two elements, compare them directly to get min and max.
3. Otherwise, divide the array into two halves at $\text{mid} = (\text{low} + \text{high}) / 2$.
4. Recursively find (min, max) of the left half and the right half.
5. Combine: overall min = smaller of the two mins; overall max = larger of the two maxes.
6. Return the combined (min, max) pair.

PROGRAM (C)

```
#include <stdio.h>

struct Pair { int minVal, maxVal; };

struct Pair minMax(int a[], int low, int high) {
    struct Pair result, left, right;
    if (low == high) {
        result.minVal = result.maxVal = a[low];
        return result;
    }
    if (high == low + 1) {
        if (a[low] < a[high]) { result.minVal = a[low]; result.maxVal = a[high]; }
        else { result.minVal = a[high]; result.maxVal = a[low]; }
        return result;
    }
    int mid = (low + high) / 2;
    left = minMax(a, low, mid);
    right = minMax(a, mid + 1, high);

    result.minVal = (left.minVal < right.minVal) ? left.minVal : right.minVal;
    result.maxVal = (left.maxVal > right.maxVal) ? left.maxVal : right.maxVal;
    return result;
}
```

```

}

int main() {
    int n;
    printf("Enter number of prices: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter values: ");
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);

    struct Pair p = minMax(a, 0, n - 1);
    printf("Minimum Stock Price = %d\n", p.minVal);
    printf("Maximum Stock Price = %d\n", p.maxVal);
    return 0;
}

```

INPUT

6
450 520 480 610 430 590

OUTPUT

Minimum Stock Price = 430
Maximum Stock Price = 610

RESULT

The Divide and Conquer min-max algorithm correctly reported the lowest and highest stock prices for the given week.

Program 9: Fastest and Slowest Race Times (Divide and Conquer)

AIM

To find the fastest (minimum) and slowest (maximum) race completion times using the Divide and Conquer approach.

ALGORITHM

1. If the segment has one element, it is both the min and max.
2. If the segment has two elements, compare them directly to get min and max.
3. Otherwise, divide the array into two halves at $\text{mid} = (\text{low} + \text{high}) / 2$.
4. Recursively find (min, max) of the left half and the right half.
5. Combine: overall min = smaller of the two mins; overall max = larger of the two maxes.
6. Return the combined (min, max) pair.

PROGRAM (C)

```

#include <stdio.h>

struct Pair { int minVal, maxVal; };

struct Pair minMax(int a[], int low, int high) {
    struct Pair result, left, right;
    if (low == high) {
        result.minVal = result.maxVal = a[low];
        return result;
    }
    if (high == low + 1) {

```

```

    if (a[low] < a[high]) { result.minVal = a[low]; result.maxVal = a[high]; }
    else { result.minVal = a[high]; result.maxVal = a[low]; }
    return result;
}
int mid = (low + high) / 2;
left = minMax(a, low, mid);
right = minMax(a, mid + 1, high);

result.minVal = (left.minVal < right.minVal) ? left.minVal : right.minVal;
result.maxVal = (left.maxVal > right.maxVal) ? left.maxVal : right.maxVal;
return result;
}

int main() {
    int n;
    printf("Enter number of runners: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter values: ");
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);

    struct Pair p = minMax(a, 0, n - 1);
    printf("Fastest Time = %d\n", p.minVal);
    printf("Slowest Time = %d\n", p.maxVal);
    return 0;
}

```

INPUT

9
14 12 18 11 15 13 17 16 10

OUTPUT

Fastest Time = 10
Slowest Time = 18

RESULT

The program correctly determined the fastest and slowest race times among all competitors using the min-max Divide and Conquer method.

Program 10: Largest and Smallest Building Heights (Divide and Conquer)

AIM

To determine the tallest and shortest building heights in a city block using the Divide and Conquer min-max algorithm.

ALGORITHM

1. If the segment has one element, it is both the min and max.
2. If the segment has two elements, compare them directly to get min and max.
3. Otherwise, divide the array into two halves at $\text{mid} = (\text{low} + \text{high}) / 2$.
4. Recursively find (min, max) of the left half and the right half.
5. Combine: overall min = smaller of the two mins; overall max = larger of the two maxes.
6. Return the combined (min, max) pair.

PROGRAM (C)

```

#include <stdio.h>

struct Pair { int minVal, maxVal; };

struct Pair minMax(int a[], int low, int high) {
    struct Pair result, left, right;
    if (low == high) {
        result.minVal = result.maxVal = a[low];
        return result;
    }
    if (high == low + 1) {
        if (a[low] < a[high]) { result.minVal = a[low]; result.maxVal = a[high]; }
        else { result.minVal = a[high]; result.maxVal = a[low]; }
        return result;
    }
    int mid = (low + high) / 2;
    left = minMax(a, low, mid);
    right = minMax(a, mid + 1, high);

    result.minVal = (left.minVal < right.minVal) ? left.minVal : right.minVal;
    result.maxVal = (left.maxVal > right.maxVal) ? left.maxVal : right.maxVal;
    return result;
}

int main() {
    int n;
    printf("Enter number of buildings: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter values: ");
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);

    struct Pair p = minMax(a, 0, n - 1);
    printf("Minimum Height = %d\n", p.minVal);
    printf("Maximum Height = %d\n", p.maxVal);
    return 0;
}

```

INPUT

```

10
120 150 98 175 140 165 110 190 130 145

```

OUTPUT

```

Minimum Height = 98
Maximum Height = 190

```

RESULT

The Divide and Conquer algorithm correctly identified the shortest and tallest buildings in the given block.